Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😁

alkemy

Clase grabada

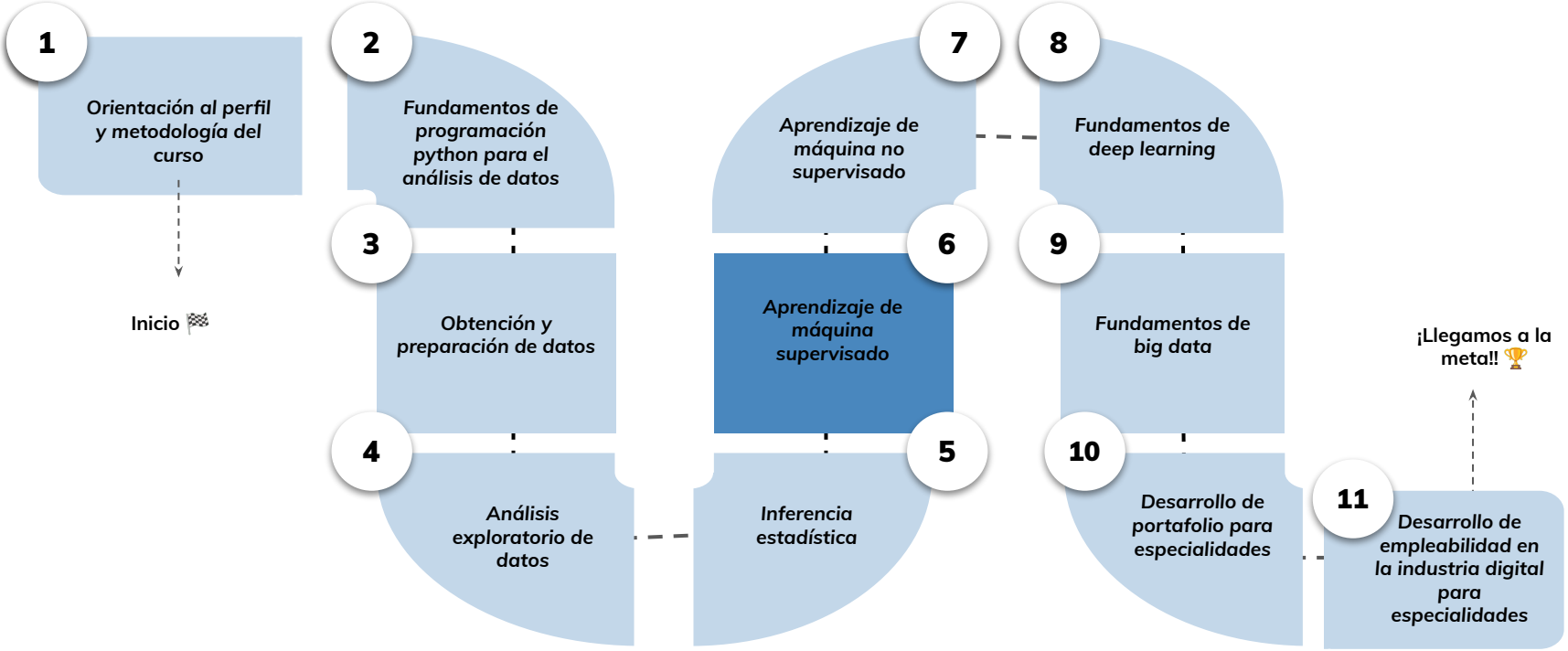
Algoritmos de clasificación- Parte I

Aprendizaje Esperado 5: Elaborar un modelo predictivo aplicando algoritmos de clasificación para resolver un problema dado de acuerdo a las buenas prácticas de la industria.

alkemy

Hoja de ruta

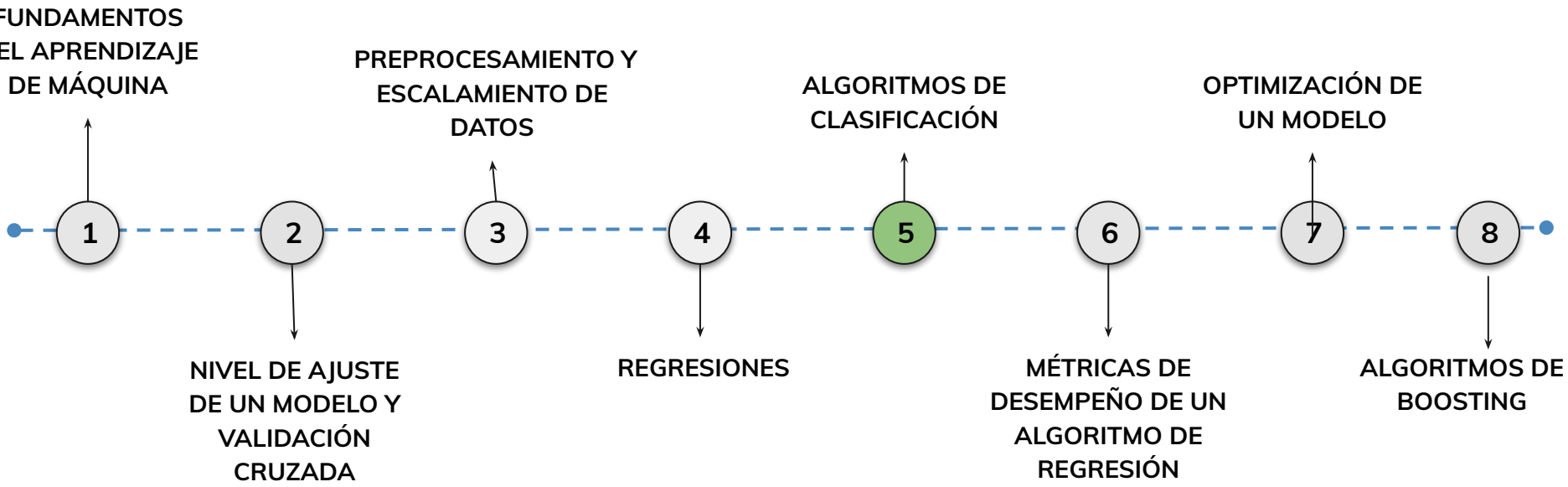
¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



alkemy

Roadmap de lecciones

*¿Cuáles **lecciones** estaremos estudiando en este módulo?*



alkemy

Learning Path

¿Cuáles temas trabajaremos hoy?

5.

Algoritmos de clasificación

K-Nearest Neighbors (K-NN) para resolver problemas de clasificación, considerando buenas prácticas de preprocesamiento y evaluación.

Fundamentos del algoritmo K-NN

- Qué es y cómo funciona el algoritmo K-NN?
- Principios clave: distancia euclideana, valor de K, ventajas y desventajas

Implementación y evaluación de K-NN en Python

- Preprocesamiento, escalamiento y entrenamiento del modelo
- Evaluación con métricas, selección de K óptimo, matriz de confusión

Objetivos de aprendizaje

¿Qué aprenderás?

- Comprender el funcionamiento del algoritmo K-Nearest Neighbors (K-NN).
- Aplicar técnicas de preprocesamiento previas a la clasificación.
- Analizar el impacto del valor de K y la métrica de distancia.
- Implementar y evaluar modelos de clasificación con Scikit-Learn.
- Interpretar los resultados mediante métricas de desempeño.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos 📚:

→ *Modelos de regresión lineales y no lineales.*

→ *Técnicas de regularización (Ridge, Lasso, ElasticNet).*

→ *Métricas para evaluar regresiones: MSE, MAE, R^2.*

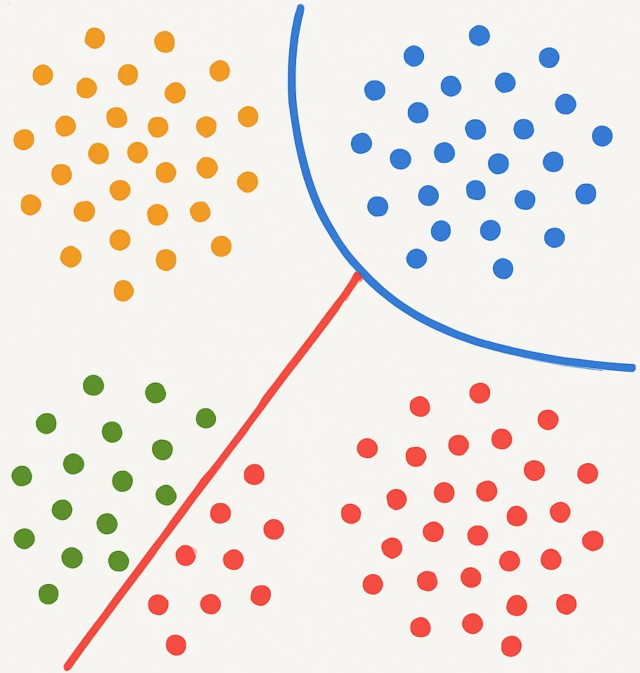
→ *Implementación práctica y validación cruzada.*

alkemy

Algoritmos de clasificación

ALGORITMOS DE CLASIFICACIÓN

Los algoritmos de clasificación son herramientas fundamentales en el campo del aprendizaje automático supervisado. Su principal objetivo es asignar una etiqueta o categoría a una observación desconocida, basándose en patrones aprendidos a partir de un conjunto de datos etiquetados.

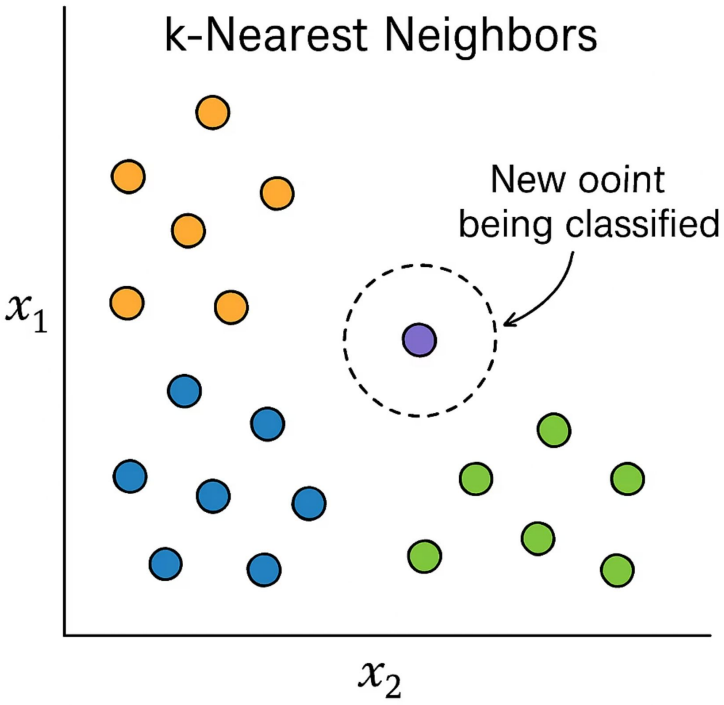


alkemy

Introducción

Entre los algoritmos más sencillos y efectivos para este propósito se encuentra el algoritmo K-Nearest Neighbors (K-NN). K-NN es un algoritmo basado en la proximidad y la similitud entre observaciones.

Su principio es sencillo: para clasificar un nuevo dato, el algoritmo busca los k datos más cercanos a él en el conjunto de entrenamiento y asigna la clase más común entre ellos. Este enfoque hace que K-NN sea un algoritmo intuitivo, fácil de implementar y eficaz en múltiples escenarios.



alkemy

Contenido del Manual

Funcionamiento de *K*-NN

Exploraremos el funcionamiento detallado del algoritmo K-NN y sus principios básicos.

Conceptos Clave

Analizaremos el escalamiento de datos y la distancia euclideana como elementos fundamentales.

Selección de Parámetros

Estudiaremos la importancia de seleccionar un valor adecuado para k y su impacto en el modelo.

Implementación Práctica

Implementaremos un modelo K-NN utilizando la librería Scikit-Learn con buenas prácticas.

Aprendizaje Esperado

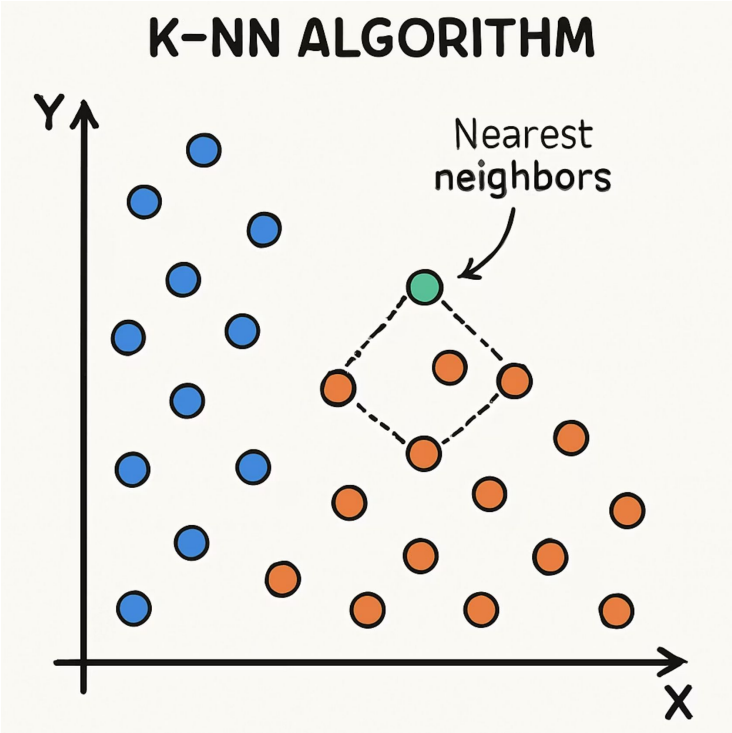
Cuando finalices la lección serás capaz de:

- Comprender en qué consiste el algoritmo K-Nearest Neighbors.
- Explicar la importancia del escalamiento de datos para el uso de K-NN.
- Interpretar el concepto de distancia euclideana y su rol en K-NN.
- Determinar la manera adecuada de seleccionar el valor de k y evaluar la exactitud del modelo.
- Identificar las ventajas y desventajas de utilizar el algoritmo K-NN.
- Implementar un clasificador K-NN utilizando la librería Scikit-Learn en Python.
- Evaluar el desempeño del modelo mediante métricas adecuadas.

¿En qué consiste el algoritmo K-Nearest Neighbors (K-NN)?

El algoritmo K-Nearest Neighbors (K-NN) es un método de clasificación supervisada que asigna una categoría a una nueva observación basándose en las clases de sus vecinos más cercanos.

Es un algoritmo de tipo lazy learning (aprendizaje perezoso), ya que no construye un modelo explícito durante el entrenamiento, sino que almacena los datos y realiza los cálculos en el momento de la predicción.



alkemy

Funcionamiento de K-NN

Almacenamiento de datos

El algoritmo almacena todos los datos del conjunto de entrenamiento.

Cálculo de distancias

Ante una nueva observación, calcula la distancia entre esta y todos los datos del conjunto de entrenamiento.

Identificación de vecinos

Identifica los k elementos más cercanos según la distancia calculada.

Asignación de clase

Asigna la clase más común entre los k vecinos más cercanos.

Principios Fundamentales de K-NN

Premisa básica

El algoritmo K-NN se basa en la premisa de que las observaciones cercanas entre sí tienden a pertenecer a la misma clase.

Algoritmo no paramétrico

K-NN es un algoritmo no paramétrico, lo que significa que no hace suposiciones explícitas sobre la distribución de los datos.

Entrenamiento rápido

El entrenamiento de K-NN es muy rápido, ya que consiste simplemente en almacenar el conjunto de datos.

Sensibilidad a la escala

K-NN es sensible a la escala de las variables, por lo que es necesario aplicar técnicas de escalamiento de datos antes de utilizar este algoritmo.

Consideraciones Importantes

La precisión del modelo dependerá de la distribución y calidad de los datos de entrenamiento, así como de la elección adecuada de la métrica de distancia y el valor de k.

El costo computacional de la predicción puede ser elevado, especialmente para conjuntos de datos grandes, ya que requiere calcular la distancia entre la nueva observación y todos los datos existentes.

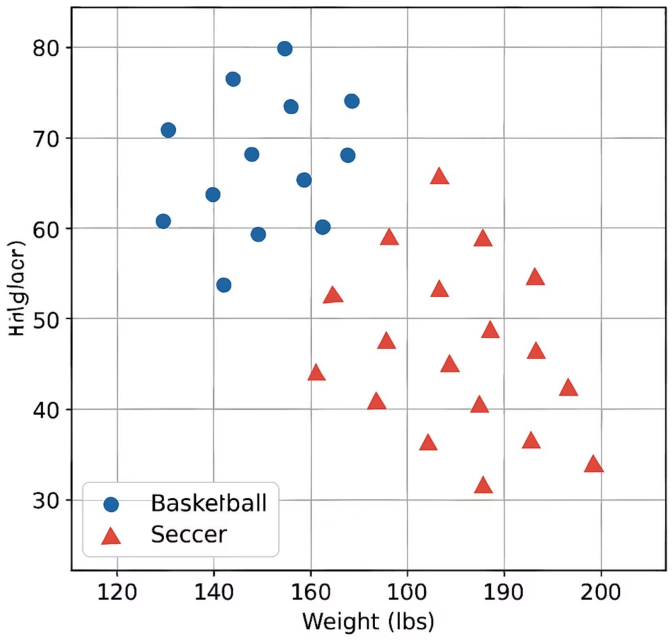
K-NN puede verse afectado por la presencia de variables irrelevantes o ruidosas, ya que estas pueden distorsionar la medición de distancia y reducir la precisión del modelo.

Es recomendable realizar un análisis exploratorio y una selección adecuada de características antes de aplicar K-NN.

Ejemplo Práctico de K-NN

Supongamos que tenemos el siguiente conjunto de datos:

ID	Altura (cm)	Peso (kg)	Deporte favorito
1	180	75	Fútbol
2	165	65	Baloncesto
3	170	70	Fútbol
4	160	60	Baloncesto



Queremos predecir el deporte favorito de una persona con altura 172 cm y peso 72 kg.

K-NN calculará la distancia de esta nueva observación con las existentes, buscará los k vecinos más cercanos (por ejemplo, k = 3) y asignará la clase más común.

En este caso, es probable que la predicción sea Fútbol, ya que los vecinos más cercanos tienen esa categoría.

alkemy

El concepto de escalamiento de datos

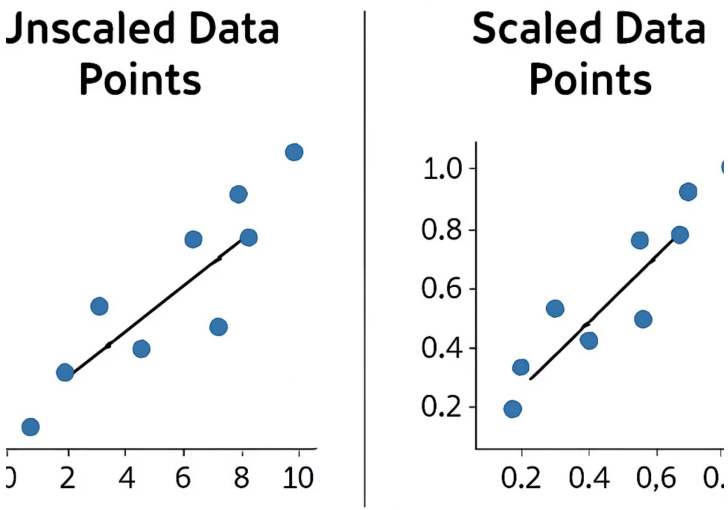
El escalamiento de datos es una técnica fundamental en el preprocesamiento de información para algoritmos de aprendizaje automático, especialmente en aquellos que dependen de la medición de distancias, como K-Nearest Neighbors (K-NN).

El objetivo del escalamiento es normalizar las variables numéricas para que todas tengan un rango similar y ninguna domine el cálculo de la distancia.

Importancia del Escalamiento en K-NN

Cuando las variables de un conjunto de datos tienen escalas muy diferentes, el cálculo de distancias puede verse distorsionado.

Por ejemplo, si una variable mide altura en centímetros (con valores entre 150 y 200) y otra mide ingresos anuales (con valores entre 10,000 y 100,000), la segunda variable tendrá un impacto mucho mayor en la distancia, incluso si es menos relevante para la clasificación.



alkemy

Técnicas de Escalamiento

1

Normalización Min-Max

Transforma los valores de cada variable para que estén en un rango específico, generalmente entre 0 y 1.

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

2

Estandarización (Z-Score)

Transforma los datos para que tengan una media de 0 y una desviación estándar de 1.

$$X_{std} = \frac{X - \mu}{\sigma}$$

donde μ es la media y σ la desviación estándar de la variable.

alkemy

Consideraciones sobre el Escalamiento

- La elección entre normalización y estandarización depende del algoritmo y la naturaleza de los datos.
- En el caso de K-NN, ambas técnicas son válidas, ya que garantizan que todas las variables contribuyan equitativamente al cálculo de la distancia.
- Es importante aplicar el escalamiento después de dividir los datos en conjunto de entrenamiento y prueba para evitar data leakage.
- El escalador debe ser ajustado únicamente con los datos de entrenamiento y luego aplicado tanto a los datos de entrenamiento como a los de prueba.
- El escalamiento también facilita la convergencia de algunos algoritmos y mejora la visualización de los datos en gráficos y análisis exploratorios.

Ejemplo Práctico de Escalamiento

Supongamos que tenemos los siguientes datos:

Altura (cm)	Peso (kg)
180	75
165	65
170	70
160	60

Aplicamos Min-Max Scaling y estandarización utilizando Scikit-

```
from sklearn.preprocessing import MinMaxScaler, StandardScalerimport numpy as np# Datos originalesX = np.array([[180, 75], [165, 65], [170, 70], [160, 60]])# Min-Max Scalingmin_max_scaler = MinMaxScaler()X_minmax = min_max_scaler.fit_transform(X)# Estandarizaciónstd_scaler = StandardScaler()X_std = std_scaler.fit_transform(X)print("Datos originales:\n", X)print("\nDatos normalizados (Min-Max):\n", X_minmax)print("\nDatos estandarizados (Z-Score):\n", X_std)
```

alkemy

Salida del Ejemplo de Escalamiento

```
Datos originales: [[180 75] [165 65] [170 70] [160 60]]Datos normalizados (Min-Max): [[1.   1.  ] [0.25
0.33] [0.5  0.67] [0.   0.  ]]Datos estandarizados (Z-Score): [[ 1.26  1.26] [-0.42 -0.42] [ 0.42
0.42] [-1.26 -1.26]]
```

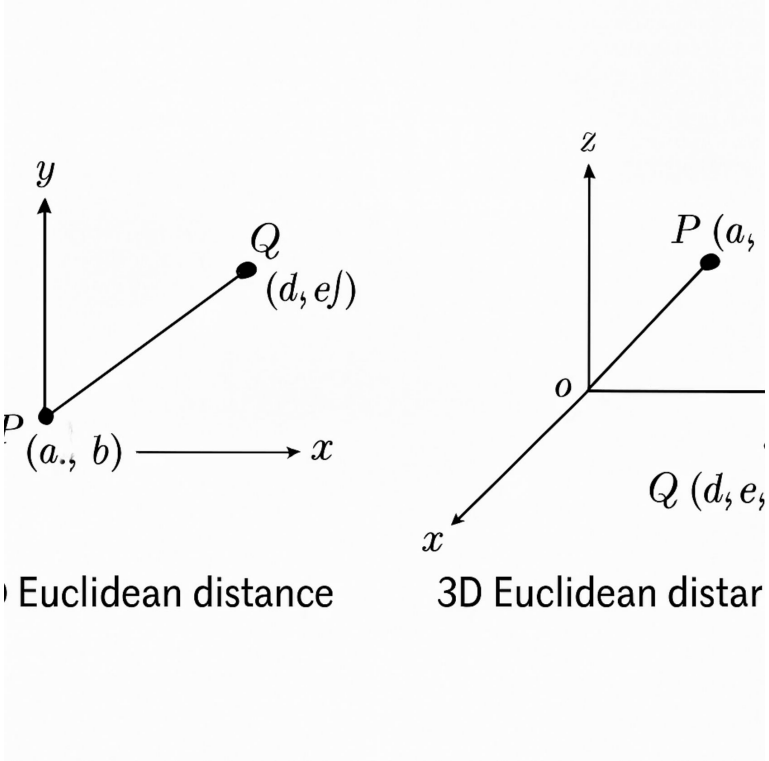
Observamos cómo ambas técnicas transforman los datos a escalas comparables, pero con diferentes rangos y distribuciones.

alkemy

El concepto de distancia euclideana

La distancia euclideana es una de las métricas más utilizadas para calcular la separación entre dos puntos en un espacio n-dimensional. Su simplicidad y significado geométrico la convierten en la opción predeterminada en muchos algoritmos de aprendizaje automático basados en proximidad, como el algoritmo K-Nearest Neighbors (K-NN).

Esta distancia mide la longitud del segmento de línea recta que conecta dos puntos.



alkemy

Fórmula de la Distancia Euclideana

En un espacio bidimensional, la fórmula de la distancia euclideana entre dos puntos $A=(x_1,y_1)$ y $B=(x_2,y_2)$ es:

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

En espacios de más dimensiones, la fórmula se extiende sumando las diferencias al cuadrado de todas las variables.

$$d(A, B) = \sqrt{\sum_{i=1}^{n} (B_i - A_i)^2}$$

alkemy

Características de la Distancia Euclideana

Significado Geométrico

Tiene un significado geométrico intuitivo y es invariante ante traslaciones y rotaciones del espacio.

Uso en *K*-NN

En el algoritmo K-NN, se utiliza para determinar cuáles son los vecinos más cercanos a una nueva observación.

Sensibilidad a la Escala

Puede verse afectada por la escala de las variables, por lo que es imprescindible aplicar técnicas de escalamiento.

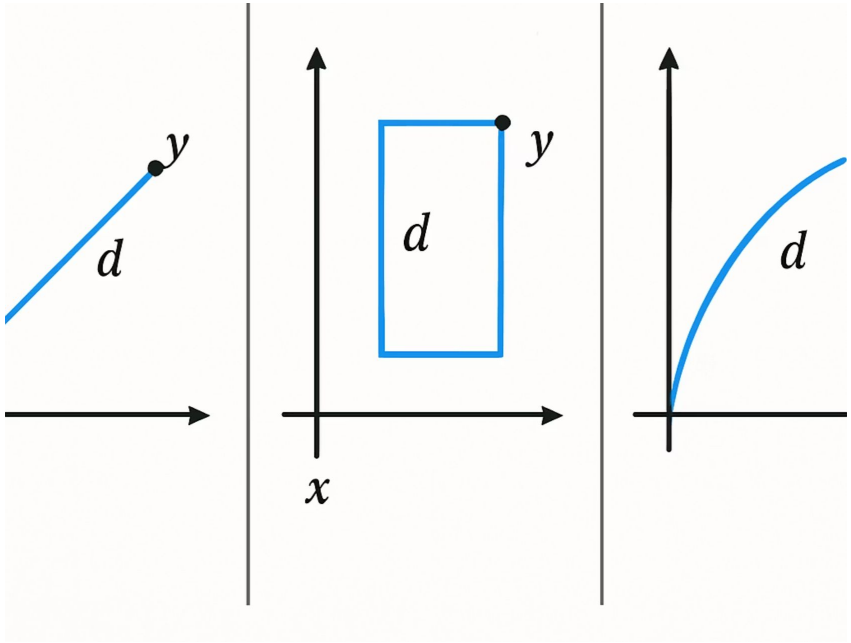
Limitaciones

No es adecuada para variables categóricas, ya que solo puede calcularse con variables numéricas.

Otras Métricas de Distancia

Además de la distancia euclideana, existen otras métricas que pueden ser consideradas según la naturaleza de los datos:

- Distancia Manhattan (o distancia L1)
- Distancia de Minkowski
- Distancia de Mahalanobis
- Distancia de Hamming (para datos binarios)



alkemy

Ejemplo Práctico de Distancia Euclideana

Supongamos que tenemos los siguientes dos puntos:

A = (180, 75)

B = (170, 70)

La distancia euclideana entre ambos es:

$$d(A, B) = \sqrt{(170 - 180)^2 + (70 - 75)^2} = \sqrt{100 + 25} = \sqrt{125} \approx 11.18$$

En Python, podemos calcularlo fácilmente:

```
import numpy as np# Definimos los puntosA = np.array([180, 75])B = np.array([170, 70])# Calculamos la distancia euclideanadistancia = np.sqrt(np.sum((A - B)**2))print(f"La distancia euclideana entre A y B es: {distancia}")
```

Salida:

```
La distancia euclideana entre A y B es: 11.180339887498949
```

alkemy

Interpretación de la Distancia Euclideana en K-NN

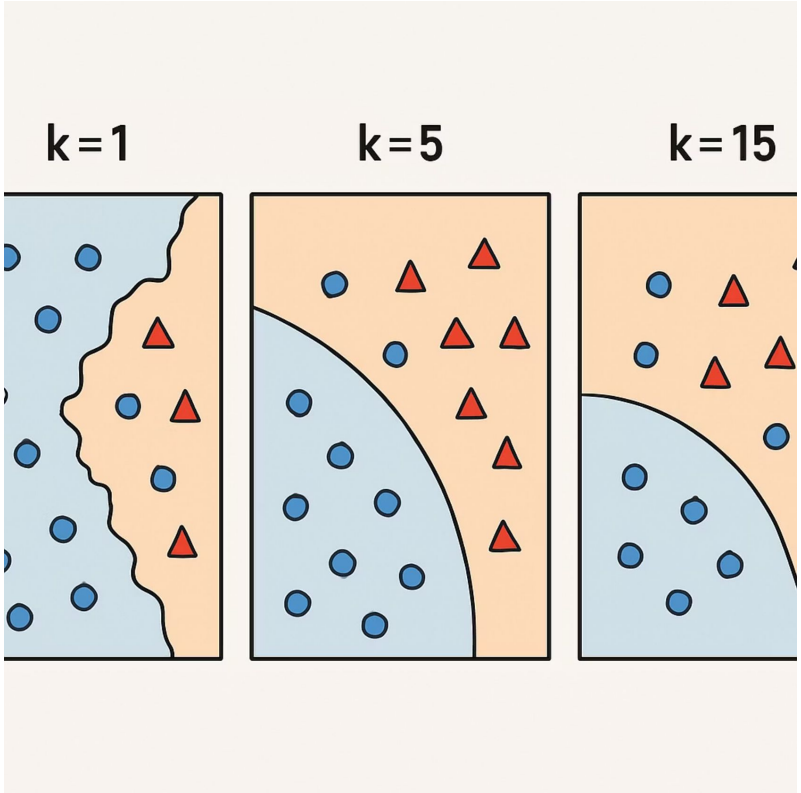
Este valor indica qué tan cercanos están ambos puntos. El algoritmo K-NN utilizará este cálculo para decidir qué clase asignar a una nueva observación, basándose en los vecinos con distancia mínima.

Cuanto menor sea la distancia euclideana entre dos puntos, más similares se consideran en el espacio de características.

Selección del valor de K y la exactitud

Uno de los elementos clave en el algoritmo K-Nearest Neighbors (K-NN) es la correcta selección del valor de K, es decir, la cantidad de vecinos que se tomarán en cuenta para determinar la clase de una nueva observación.

La elección de este parámetro tiene un impacto directo en la precisión y la capacidad de generalización del modelo.



alkemy

Impacto del Valor de K

K pequeño (ej. K = 1)

El modelo puede ser muy sensible a los datos de entrenamiento, incluyendo posibles errores o ruido. Esto genera un problema conocido como sobreajuste (overfitting).

K grande

El modelo pierde sensibilidad y puede cometer errores al incluir vecinos que no son relevantes para la clasificación. Este fenómeno se conoce como subajuste (underfitting).

K óptimo

No existe un valor de K óptimo universal. La elección adecuada depende de la naturaleza y tamaño del conjunto de datos.

Estrategias para Seleccionar K

Prueba de diferentes valores

Un enfoque común es probar diferentes valores de K y seleccionar aquel que minimice el error en un conjunto de validación o mediante técnicas de validación cruzada.

Evaluación de exactitud

Evaluar el porcentaje de exactitud (accuracy) para cada valor posible de K y graficar el resultado.

Identificación del punto óptimo

Identificar el punto donde la exactitud es máxima y evitar tanto el sobreajuste como el subajuste.

Análisis de métricas adicionales

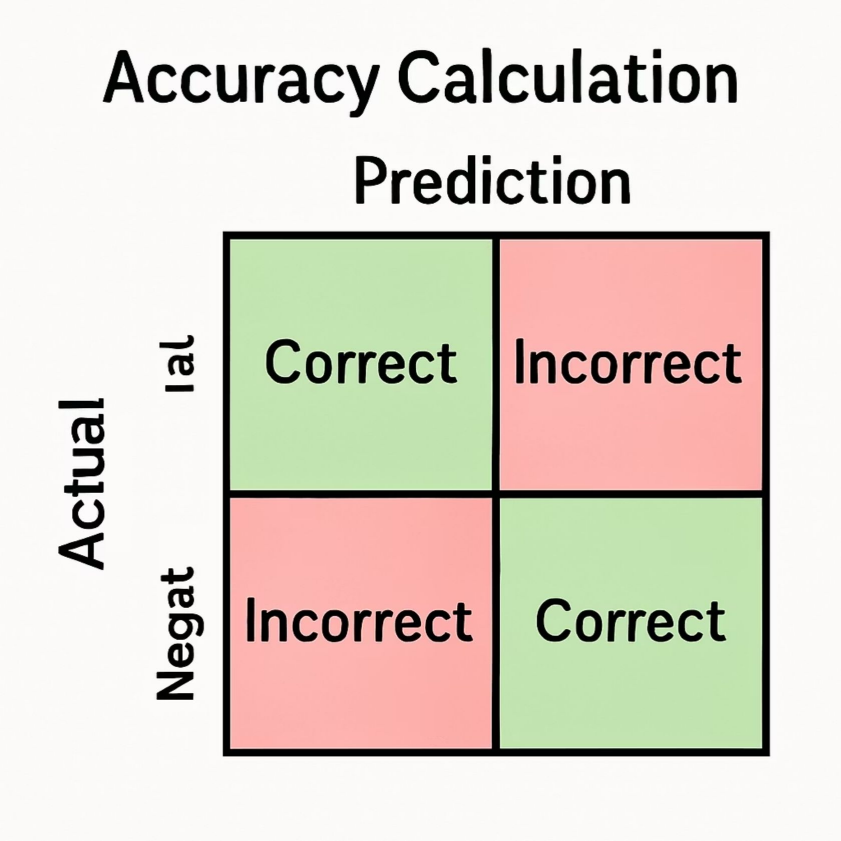
Analizar otras métricas de evaluación, como la matriz de confusión, la precisión, la sensibilidad y la puntuación F1.

Cálculo de la Exactitud

El cálculo de la exactitud consiste en comparar las predicciones del modelo con las etiquetas reales del conjunto de prueba y calcular el porcentaje de aciertos.

$$Exactitud = \frac{Predicciones\ correctas}{Total\ de\ predicciones}$$

Esta métrica es adecuada cuando las clases están balanceadas.



alkemy

Ejemplo Práctico de Selección de K

Utilizando el conjunto de datos de altura y peso, podemos probar diferentes valores de K y observar el efecto en la exactitud:

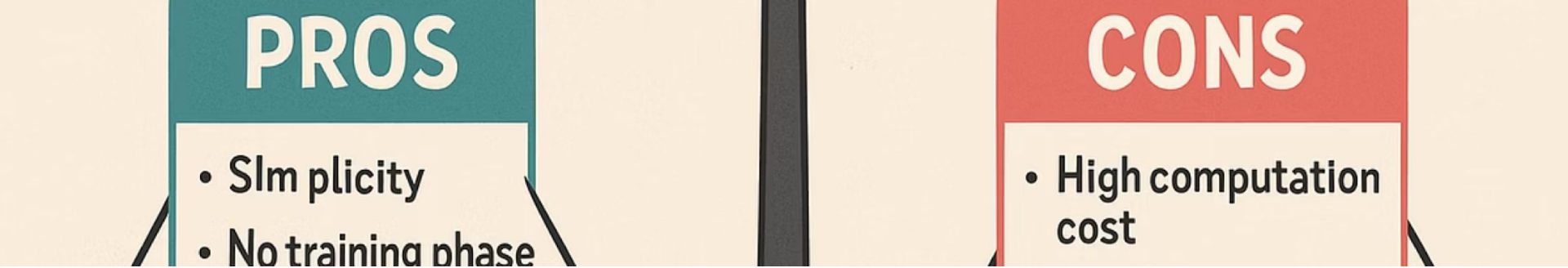
```
import numpy as npimport matplotlib.pyplot as pltfrom sklearn.neighbors import KNeighborsClassifierfrom sklearn.model_selection import
train_test_splitfrom sklearn.preprocessing import StandardScalerfrom sklearn.metrics import accuracy_score# Datos de ejemploX = np.array([[180,
75], [165, 65], [170, 70], [160, 60],  [175, 72], [162, 63], [178, 74], [168, 68]])y = np.array(['Fútbol', 'Baloncesto', 'Fútbol', 'Baloncesto',
'Fútbol', 'Baloncesto', 'Fútbol', 'Baloncesto'])# Dividir datos y escalarX_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42)scaler = StandardScaler()X_train_scaled = scaler.fit_transform(X_train)X_test_scaled = scaler.transform(X_test)# Probar diferentes
valores de Kk_values = range(1, 6)accuracies = []for k in k_values: knn = KNeighborsClassifier(n_neighbors=k) knn.fit(X_train_scaled, y_train)
y_pred = knn.predict(X_test_scaled) accuracies.append(accuracy_score(y_test, y_pred))# Graficar resultadosplt.figure(figsize=(10,
6))plt.plot(k_values, accuracies, marker='o')plt.title('Exactitud vs. Valor de K')plt.xlabel('Valor de
K')plt.ylabel('Exactitud')plt.grid(True)plt.show()
```

Gráfico de Exactitud vs. Valor de K

Este gráfico permite observar cómo varía la exactitud según el valor de K y ayuda a elegir el más adecuado para el conjunto de datos.

En este ejemplo, podríamos identificar el valor de K que maximiza la exactitud y utilizarlo para nuestro modelo final.

alkemy



Ventajas y desventajas de utilizar el algoritmo

El algoritmo K-Nearest Neighbors (K-NN) es uno de los más sencillos y utilizados en tareas de clasificación supervisada. Su simplicidad y naturaleza intuitiva hacen que sea una opción atractiva para problemas en los que se requiere rapidez y facilidad de implementación.

Ventajas de K-NN

Simplicidad

Fácil de implementar e interpretar, lo que lo hace accesible incluso para principiantes en aprendizaje automático.

Entrenamiento Rápido

No requiere un proceso complejo de entrenamiento. El algoritmo simplemente almacena los datos y realiza los cálculos en el momento de la predicción.

Flexibilidad

Es un algoritmo no paramétrico, lo que significa que no hace suposiciones explícitas sobre la distribución de los datos.

Multiclase

Puede adaptarse fácilmente a problemas multicategoría y no requiere ajustes significativos para manejar múltiples clases.

Desventajas de K-NN

Costo Computacional

Alto coste computacional en la fase de predicción. El tiempo de predicción aumenta considerablemente cuando el conjunto de datos es grande.

Sensibilidad a la Escala

El rendimiento es muy sensible a la escala de las variables. Requiere normalización o estandarización previa.

Vulnerabilidad al Ruido

Es vulnerable a la presencia de ruido y variables irrelevantes, ya que todas las variables contribuyen al cálculo de la distancia.

Selección de K

La elección del valor de K puede ser un desafío. Un valor inadecuado puede llevar a sobreajuste o subajuste.

Resumen de Ventajas y Desventajas

Ventajas	Desventajas
Fácil de implementar e interpretar	Coste computacional alto en predicción
No requiere entrenamiento previo	Sensible a la escala de las variables
No paramétrico (no hace suposiciones sobre los datos)	Afectado por ruido y variables irrelevantes
Admite múltiples clases	Requiere selección cuidadosa del valor de K
Flexibilidad para incorporar nuevos datos	No apto para conjuntos de datos muy grandes

El conocimiento de estas ventajas y desventajas es fundamental para decidir cuándo y cómo aplicar el algoritmo K-NN en un proyecto de aprendizaje automático.

Live Coding

¿En qué consistirá la Demo?

Implementaremos paso a paso un clasificador K-NN para predecir el deporte favorito de personas según altura y peso. Utilizaremos técnicas de escalamiento, selección de K óptimo y métricas de evaluación.

1. *Preprocesamiento del dataset.*
2. *Aplicación de StandardScaler y MinMaxScaler.*
3. *Implementación de KNeighborsClassifier.*
4. *Evaluación de precisión para diferentes valores de K.*
5. *Visualización de exactitud con Matplotlib.*
6. *Uso de matriz de confusión y F1-score.*
7. *Elección del modelo final.*
8. *Predicción con datos nuevos.*

Tiempo: 25 Minutos

Momento:

Time-out!

⏰5 -10 min.

alkemy

Ejercicio

Clasificando clientes bancarios según su comportamiento

Clasificando clientes bancarios según su comportamiento

Contexto: 🙌

Un banco quiere predecir si un cliente aprobaría o no un producto financiero (por ejemplo, un préstamo), en base a sus características como edad, ingresos, estado civil, educación, etc. Tu tarea sera construir un modelo de clasificación usando K-NN que ayude al área comercial a tomar mejores decisiones.

Consigna: ✍️

1. Cargar el dataset clientes_banco.csv.
2. Realizar una exploración inicial de los datos.
3. Aplicar escalamiento a las variables numéricas.
4. Convertir variables categóricas en variables numéricas (One Hot Encoding).
5. Dividir en conjunto de entrenamiento y prueba.
6. Entrenar un modelo K-NN con varios valores de K.
7. Calcular accuracy, matriz de confusión y F1-score para cada valor.
8. Graficar el impacto de K en el rendimiento del modelo.
9. Elegir el mejor K y explicar el por qué.
10. Realizar una predicción para un cliente nuevo.

Tiempo 🕛**:** *45 minutos*

alkemy

Clasificando clientes bancarios según su comportamiento

Paso a paso: ⚙️

1. Importar librerías necesarias (pandas, numpy, sklearn).
2. Limpiar y escalar datos.
3. Usar train_test_split para separar los conjuntos.
4. Entrenar con KNeighborsClassifier.
5. Evaluar con classification_report y confusion_matrix.
6. Graficar la precisión para valores de K entre 1 y 20.
7. Seleccionar el modelo final y guardar.
8. Usar .predict() para simular una predicción.

¿Alguna consulta?

alkemy

Resumen

¿Qué logramos en esta clase?

- ✓ ***Comprendimos el funcionamiento del algoritmo K-NN.***
- ✓ ***Aplicamos técnicas de escalamiento y procesamiento.***
- ✓ ***Evaluamos distintos valores de K con métricas específicas.***
- ✓ ***Implementamos un clasificador completo en Python.***

¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las ***actividades disponibles en la plataforma.*** *Estas propuestas son clave para afianzar lo trabajado y* ***forman parte obligatoria del recorrido de aprendizaje.***

💪 Análisis de caso ——— 💪 Selección Múltiple

💪 Comprensión lectora

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

alkemy

< ¡Muchas gracias! >

alkemy